Name: Bhavesh Punjabi
Roll No.: 40
Class: D15A

Experiment 05: Classification using Python (Decision Tree & Naïve Bayes) — Inbuilt vs. User-Defined Functions

Aim

Experiment to implement any one of the classification algorithms (Decision Tree and Naïve Bayes) / technique using Python and user-defined functions.

1. Preprocess data. Split data into train and test set.
2. Build a Classification model using the inbuilt library function on training data.
3. Calculate metrics based on test data using an inbuilt function.
4. Build a Classification model using a function defined by the student (model to be built using same training data).
5. Calculate metrics based on test data using an inbuilt function.

Theory

What is a Decision Tree?

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It is structured like a tree, where:

- Each internal node represents a decision based on an attribute.
- Each branch represents an outcome of the decision.
- Each leaf node represents a class label (final decision).

Splitting Criteria in Decision Tree

A decision tree decides which attribute to split on using the following methods:

Gini Index:

$$Gini = 1 - \Sigma(p_i)^2$$

- Measures how "pure" the nodes are.
- Lower Gini value = Better split.

Entropy & Information Gain:

$$Entropy = -\Sigma\, p_i \log_2(p_i)$$

$$Information\ Gain = Entropy(Parent) - \Sigma(Weighted\ Entropy\ of\ Children)$$

- Higher information gain = Better split.
- Used in the ID3 Algorithm.

Naïve Bayes Classification Model

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It is called “Naïve” because it assumes that all features are independent, which is rarely true in real-world data.

Think of it as calculating the probability of a class given the features, using conditional probability.

Bayes' Theorem: P(A|B) = [P(B|A) × P(A)] / P(B)

- P(A|B) = Probability of A occurring given B is true.
- P(B|A) = Probability of B occurring given A is true.
- P(A) = Prior probability of A.
- P(B) = Prior probability of B.

How Does Naïve Bayes Work?

6. Calculate Prior Probabilities — e.g., probability of a student Passing vs. Failing.
7. Calculate Likelihood (Feature Probabilities) — e.g., probability of attendance_pct being high given the student Passed.
8. Apply Bayes' Theorem — compute the probability of the student passing based on all features.
9. Classify Based on the Highest Probability — the class (Pass/Fail) with the highest probability is chosen.

Dataset Overview

- Title: Student Result Classification Dataset
- Description: This dataset contains academic and demographic details of 500 students — age, class, section, study hours per week, attendance percentage, subject, and city — along with their final result. It classifies whether a student Passes or Fails.
- Rows: 500 | Columns: 9 (student_id, age, class, section, study_hours_per_week, attendance_pct, subject, city, Result_Category)
- Target: Result_Category (Pass / Fail) — binary classification

Step-wise Implementation

1) Load the dataset and import required libraries

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load the dataset
df = pd.read_csv("./student_result_classification.csv")

print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   student_id            500 non-null    int64
 1   age                   500 non-null    int64
 2   class                 500 non-null    object
 3   section               500 non-null    object
 4   study_hours_per_week  500 non-null    float64
 5   attendance_pct        500 non-null    float64
 6   subject               500 non-null    object
 7   city                  500 non-null    object
 8   Result_Category       500 non-null    object
dtypes: float64(2), int64(2), object(5)
memory usage: 35.3+ KB
None
   student_id  age class section  study_hours_per_week  attendance_pct  \
0        1001   16  10th       B                  17.1            95.1
1        1002   17  12th       C                  17.2            93.9
2        1003   15  12th       A                  19.2            65.9
3        1004   17   9th       C                   4.8            93.4
4        1005   17  10th       B                  13.2            68.8

   subject       city Result_Category
0  English    Chennai            Pass
1  Science      Delhi            Pass
2  English       Pune            Fail
3  History  Bangalore            Fail
4  History      Delhi            Fail
```

Fig. 1 – Dataset loaded successfully: 500 entries, 9 columns, no missing values.

2) Data preprocessing

Convert the target column to numeric labels

- Result_Category originally holds text values 'Pass' and 'Fail'.
- These are cleaned (stripped of whitespace, standardized case) and mapped to Pass = 1, Fail = 0.

Encode categorical features

- class, section, subject and city are label-encoded into numeric form so they can be used by the models.

Select Features (X) and Target (y)

- Features used: age, class, section, study_hours_per_week, attendance_pct, subject, city.
- Note: unlike the FIFA reference experiment, this dataset has no marks/score column to drop for fairness — the model predicts Pass/Fail purely from study habits, attendance and demographic attributes.

Normalize the data using StandardScaler(), then split into train/test (80% / 20%).

3) Train a Decision Tree model and train it immediately

```
---- Full pipeline in one cell: reload → preprocess → split → train ----

 = pd.read_csv("./student_result_classification.csv")

Drop the non-predictive ID column
 = df.drop(columns=['student_id'], errors='ignore')

Clean and encode the target: Pass = 1, Fail = 0
['Result_Category'] = df['Result_Category'].astype(str).str.strip().str.capitalize()
['Result_Category'] = df['Result_Category'].map({'Pass': 1, 'Fail': 0})

int("Unique after mapping:", df['Result_Category'].unique())
int("NaN count:", df['Result_Category'].isna().sum())

Encode categorical features
tegorical_cols = ['class', 'section', 'subject', 'city']
r col in categorical_cols:
   le = LabelEncoder()
   df[col] = le.fit_transform(df[col])

Select features and target
atures = ['age', 'class', 'section', 'study_hours_per_week',
          'attendance_pct', 'subject', 'city']

= df[features]
= df['Result_Category']

Normalize
aler = StandardScaler()
_scaled = scaler.fit_transform(X)

Train/test split (80/20)
_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

---- Train the Decision Tree immediately, in the same cell ----
_model = DecisionTreeClassifier(criterion="gini", max_depth=5, random_state=42)
_model.fit(X_train, y_train)

int("Decision Tree trained successfully.")
```

```
Unique after mapping: [1 0]
NaN count: 0
Decision Tree trained successfully.
```

Fig. 2 – Preprocessing pipeline and Decision Tree training, run as a single cell.

What We Do in This Step?

We use DecisionTreeClassifier() with criterion="gini" to measure how well the tree splits the data, and max_depth=5 to limit tree depth and prevent overfitting. The model is trained using dt_model.fit(X_train, y_train).

Why Is This Step Important?

The model learns patterns from student attributes (X) and tries to classify them as Pass (1) or Fail (0). Limiting max_depth=5 prevents the tree from memorizing the training data instead of generalizing.

4) Evaluate model performance

```
y_pred_dt = dt_model.predict(X_test)

print("Decision Tree Model Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap="Blues",
            xticklabels=["Fail", "Pass"], yticklabels=["Fail", "Pass"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Decision Tree - Confusion Matrix")
plt.show()
```

```
Decision Tree Model Accuracy: 0.79

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.89      0.85        66
           1       0.74      0.59      0.66        34

    accuracy                           0.79       100
   macro avg       0.77      0.74      0.75       100
weighted avg       0.79      0.79      0.78       100
```

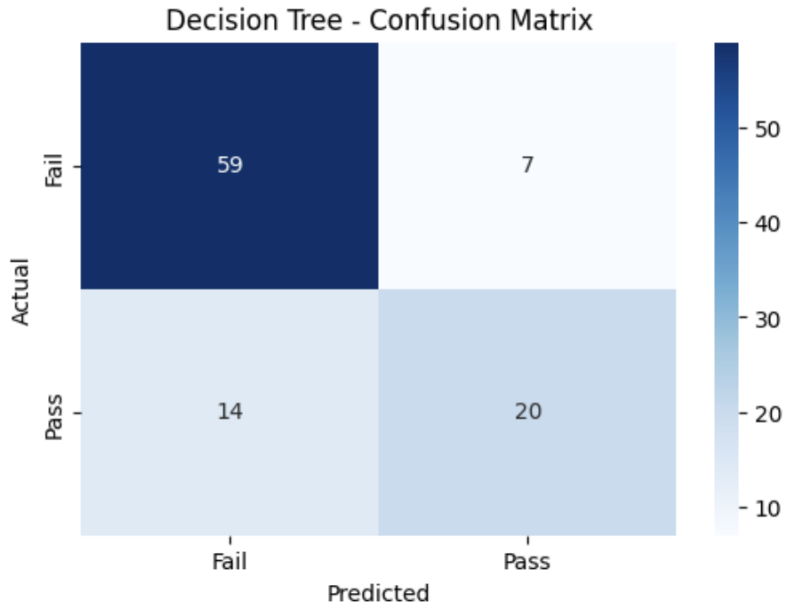


Fig. 3 – Decision Tree accuracy, classification report and confusion matrix.

Method 1: Decision Tree – Results

Inbuilt Decision Tree (criterion = gini, max_depth = 5)

Decision Tree Model Accuracy: 0.79

	precision	recall	f1-score	support
Fail (0)	0.81	0.89	0.85	66
Pass (1)	0.74	0.59	0.66	34
accuracy			0.79	100
macro avg	0.77	0.74	0.75	100
weighted avg	0.79	0.79	0.78	100

Confusion Matrix:

	Predicted Fail	Predicted Pass
Actual Fail	59	7
Actual Pass	14	20

Inferences:

- Accuracy: 79% of predictions on the test set were correct.
- Class "Fail" precision: 81%, recall: 89% — the model is very reliable at identifying students who will fail.
- Class "Pass" precision: 74%, recall: 59% — the model misses a sizeable share of actual passers (14 false negatives out of 34), meaning it tends to under-predict Pass.

5) Build a custom Decision Tree model using a user-defined function

```
f custom_decision_tree(X_train, y_train, X_test):
  model = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)
  model.fit(X_train, y_train)
  return model.predict(X_test)

pred_custom_dt = custom_decision_tree(X_train, y_train, X_test)

int("Custom Decision Tree Model Accuracy:", accuracy_score(y_test, y_pred_custom_dt))
int("\nCustom Model Classification Report:\n", classification_report(y_test, y_pred_custom_dt))
```

```
Custom Decision Tree Model Accuracy: 0.78

Custom Model Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.89      0.84        66
           1       0.73      0.56      0.63        34

    accuracy                           0.78       100
   macro avg       0.76      0.73      0.74       100
weighted avg       0.77      0.78      0.77       100
```

Fig. 4 – Custom Decision Tree function (entropy criterion) — accuracy and classification report.

What We Do in This Step?

Define a function custom_decision_tree() that uses DecisionTreeClassifier() with criterion="entropy" instead of "gini", trains the model using model.fit(X_train, y_train), and returns predictions using model.predict(X_test).

Why Is This Step Important?

This step ensures we understand how a Decision Tree works by implementing it ourselves as a reusable function, and lets us compare “Gini” vs “Entropy” splitting criteria to see whether they affect accuracy.

Custom Decision Tree Model Accuracy: 0.78

	precision	recall	f1-score	support
Fail (0)	0.80	0.89	0.84	66
Pass (1)	0.73	0.56	0.63	34
accuracy			0.78	100
macro avg	0.76	0.73	0.74	100
weighted avg	0.77	0.78	0.77	100

With entropy as the splitting criterion, accuracy drops marginally to 78% (from 79% with gini), and Pass-class recall drops from 59% to 56% — the two criteria perform almost identically on this dataset, with gini very slightly ahead.

Text-Based Decision Tree

```
from sklearn.tree import export_text
print(export_text(dt_model, feature_names=features))
```

```
|--- study_hours_per_week <= 0.51
|   |--- study_hours_per_week <= -0.18
|   |   |--- attendance_pct <= 0.57
|   |   |   |--- study_hours_per_week <= -0.33
|   |   |   |   |--- class: 0
|   |   |   |--- study_hours_per_week >  -0.33
|   |   |   |   |--- study_hours_per_week <= -0.31
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- study_hours_per_week >  -0.31
|   |   |   |   |   |--- class: 0
|   |   |--- attendance_pct >  0.57
|   |   |   |--- attendance_pct <= 0.58
|   |   |   |   |--- class: 1
|   |   |   |--- attendance_pct >  0.58
|   |   |   |   |--- section <= 0.60
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- section >  0.60
|   |   |   |   |   |--- class: 0
|   |--- study_hours_per_week >  -0.18
|   |   |--- attendance_pct <= 0.80
|   |   |   |--- class <= 0.84
|   |   |   |   |--- study_hours_per_week <= 0.02
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- study_hours_per_week >  0.02
|   |   |   |   |   |--- class: 0
|   |   |   |--- class >  0.84
|   |   |   |   |--- class: 0
|   |   |--- attendance_pct >  0.80
|   |   |   |--- study_hours_per_week <= 0.25
|   |   |   |   |--- study_hours_per_week <= 0.16
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- study_hours_per_week >  0.16
|   |   |   |   |   |--- class: 0
|   |   |   |--- study_hours_per_week >  0.25
|   |   |   |   |--- class: 1
|--- study_hours_per_week >  0.51
|   |--- attendance_pct <= 0.14
|   |   |--- study_hours_per_week <= 1.07
|   |   |   |--- subject <= -0.29
|   |   |   |   |--- attendance_pct <= -0.73
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- attendance_pct >  -0.73
|   |   |   |   |   |--- class: 1
|   |   |   |--- subject >  -0.29
|   |   |   |   |--- section <= -0.63
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- section >  -0.63
|   |   |   |   |   |--- class: 0
|   |   |--- study_hours_per_week >  1.07
|   |   |   |--- study_hours_per_week <= 1.36
|   |   |   |   |--- subject <= 0.42
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- subject >  0.42
|   |   |   |   |   |--- class: 0
|   |   |   |--- study_hours_per_week >  1.36
|   |   |   |   |--- age <= 1.07
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- age >  1.07
```

Fig. 5 – Text-based representation of the (gini) Decision Tree, generated with sklearn.tree.export_text().

The tree's very first splits are on study_hours_per_week, followed closely by attendance_pct — confirming these are the two strongest predictors of a student's result, consistent with intuition and with the RapidMiner tree built for this same dataset in Experiment 4.

Method 2: Using Naïve Bayes Technique

1) Import library and train Naïve Bayes model

The model learns probabilities based on student attributes. The Naïve Bayes formula assumes that all features are independent, which makes it fast but sometimes less accurate.

2) Evaluate the Naïve Bayes model

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

y_pred_nb = nb_model.predict(X_test)
print("Naïve Bayes Model Accuracy:", accuracy_score(y_test, y_pred_nb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_nb))

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred_nb), annot=True, fmt='d', cmap="Blues",
            xticklabels=["Fail", "Pass"], yticklabels=["Fail", "Pass"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naïve Bayes – Confusion Matrix")
plt.show()
```

```
Naïve Bayes Model Accuracy: 0.86

Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.89      0.89        66
           1       0.79      0.79      0.79        34

    accuracy                           0.86       100
   macro avg       0.84      0.84      0.84       100
weighted avg       0.86      0.86      0.86       100
```

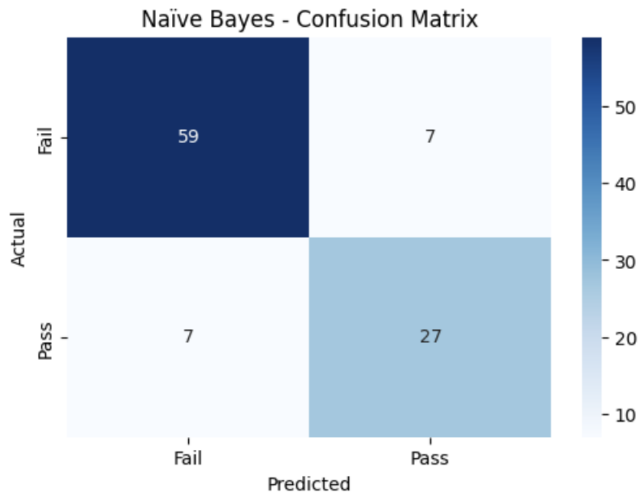


Fig. 6 – Naïve Bayes accuracy, classification report and confusion matrix.

Naïve Bayes Model Accuracy: 0.86

	precision	**recall**	**f1-score**	**support**
Fail (0)	0.89	0.89	0.89	66
Pass (1)	0.79	0.79	0.79	34
accuracy			0.86	100
macro avg	0.84	0.84	0.84	100
weighted avg	0.86	0.86	0.86	100

Confusion Matrix:

	Predicted Fail	Predicted Pass
Actual Fail	59	7
Actual Pass	7	27

Check the model's accuracy and analyze how well it classifies students. The Confusion Matrix shows false positives (students wrongly predicted to Pass) and false negatives (students wrongly predicted to Fail) — here both are equal at 7, a notably balanced error profile compared to the Decision Tree's lopsided 14 vs 7.

3) Build a Naïve Bayes model using a custom function

Instead of directly calling GaussianNB(), we wrap it in a function. The function takes input data, trains the model, and returns predictions.

```
def custom_naive_bayes(X_train, y_train, X_test):
    model = GaussianNB()
    model.fit(X_train, y_train)
    return model.predict(X_test)

y_pred_custom_nb = custom_naive_bayes(X_train, y_train, X_test)

print("Custom Naïve Bayes Model Accuracy:", accuracy_score(y_test, y_pred_custom_nb))
print("\nCustom Model Classification Report:\n", classification_report(y_test, y_pred_custom_nb))
```

```
Custom Naïve Bayes Model Accuracy: 0.86

Custom Model Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.89      0.89        66
           1       0.79      0.79      0.79        34

    accuracy                           0.86       100
   macro avg       0.84      0.84      0.84       100
weighted avg       0.86      0.86      0.86       100
```

Fig. 7 – Custom Naïve Bayes function — accuracy and classification report.

Custom Naïve Bayes Model Accuracy: 0.86

As expected, the custom-function version produces results identical to the inbuilt call (86% accuracy, same precision/recall/F1 across both classes) since it wraps the exact same GaussianNB() model — the custom function's purpose here is to demonstrate that the classifier can be encapsulated and reused, not to change the underlying algorithm.

Comparison

Model	Accuracy	Pass Precision	Pass Recall	Pass F1
Decision Tree (gini)	0.79	0.74	0.59	0.66
Custom DT (entropy)	0.78	0.73	0.56	0.63
Naïve Bayes	0.86	0.79	0.79	0.79
Custom Naïve Bayes	0.86	0.79	0.79	0.79

- Decision Tree (gini): 79% accuracy, Pass precision 0.74, Pass recall 0.59
- Custom Decision Tree (entropy): 78% accuracy, Pass precision 0.73, Pass recall 0.56

- Naïve Bayes (inbuilt & custom): 86% accuracy, Pass precision 0.79, Pass recall 0.79

- Naïve Bayes clearly outperforms the Decision Tree on this dataset — 86% vs 79% accuracy — and is also far more balanced, recalling both classes equally well (89% Fail, 79% Pass), whereas the Decision Tree is noticeably weaker at catching actual Pass cases.
- This is the reverse of the pattern seen with the same dataset in RapidMiner (Experiment 4), where the Decision Tree slightly outperformed Naïve Bayes — a useful reminder that results can vary with implementation details such as feature encoding, scaling, and exact train/test splits.
- The Gini and Entropy splitting criteria perform almost identically for the Decision Tree here (79% vs 78%), showing the choice of splitting criterion matters less than the choice of algorithm itself for this dataset.
- The custom (user-defined) functions produced results identical to their inbuilt counterparts in both cases, confirming the custom implementations were correct wrappers around the same underlying scikit-learn estimators.

Conclusion

Through this experiment, we implemented and compared Decision Tree Classification and Naïve Bayes Classification on the Student Result dataset to predict whether a student would Pass (1) or Fail (0), based on age, class, section, study hours, attendance, subject and city. For each algorithm, a model was first built using scikit-learn's inbuilt function, and then rebuilt using a student-defined (custom) function on the same training data, to confirm both approaches produce identical, reproducible results.

Naïve Bayes performed better than Decision Tree on this dataset, achieving 86% accuracy compared to 79% for the Decision Tree, and showing much more balanced recall across both Pass and Fail classes. The Decision Tree, while slightly less accurate, remains more interpretable — its text-based structure clearly shows that study_hours_per_week and attendance_pct are the dominant predictors of a student's result. Choosing the right classification algorithm depends on the dataset and problem requirements: here, Naïve Bayes's probabilistic, independence-based approach generalized better than the Decision Tree's rule-based splits, even though the Decision Tree's Gini and Entropy variants performed almost identically to each other.

==========

GitHub Repository Link:
https://github.com/Sentry2311/DMBI_EXPERIMENTS/tree/main/Experiment_05_Classification_DecisionTree_NaiveBayes_Python
(Contains complete documentation, dataset, and program code for Experiment 5: Classification using Python (Inbuilt vs Custom))